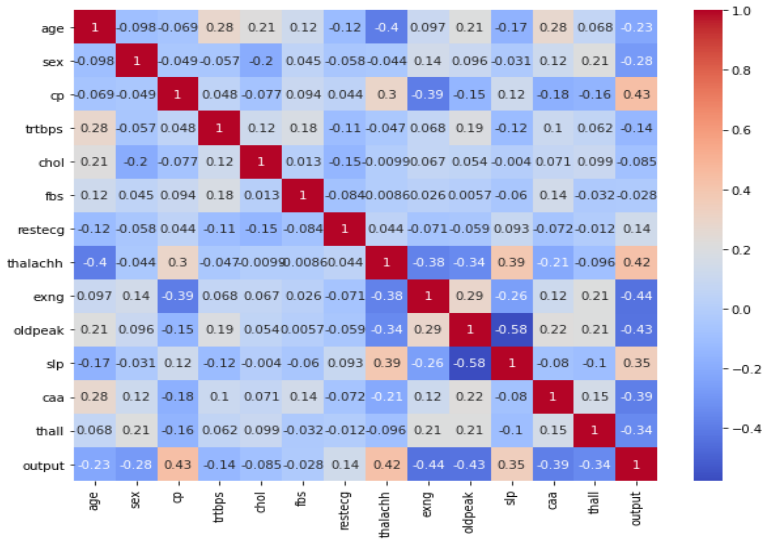
Project Title – Heart Attack Prediction

While reviewing various approaches by other people on the Heart Attack Prediction problem there were various insights which helped us gain knowledge on how to tackle this problem. In particular, Akash Patel attempt really stood out as he has done a very thorough exploratory data analysis on the dataset and also used multiple predictive and classification algorithms like Logistic Regression, Decision Tree, Random Forest, XGBoost, KNeighbours , Support Vector Machine and AdaBoost. Among these algorithms Random Forest and KNeighbours gave the highest accuracy of 90.2%. The thing that stood out the most is the various techniques used to visualize and find the most correlated features.

```
# correlation heatmap
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot = True, cmap='coolwarm')
```

```
<AxesSubplot:>
```

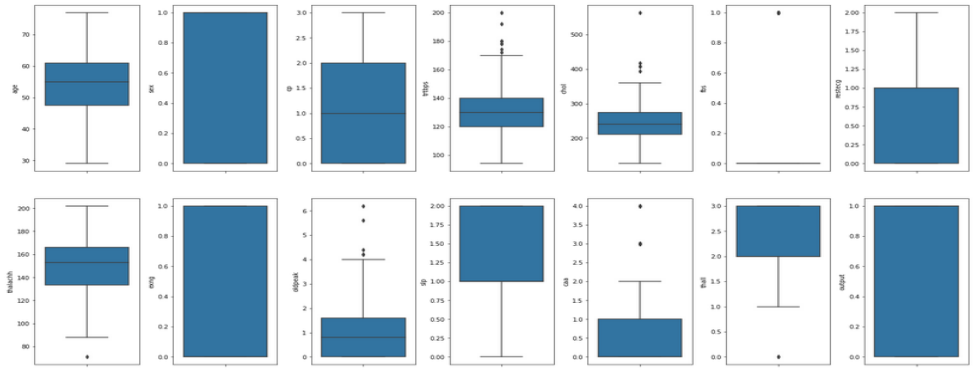


If two features are highly correlated we can drop one of them. Other important aspects such as outliers and Feature Normalization which impact the performance of an algorithm were also tackled.

```
# Visualizing dataset and also checking for outliers

fig, ax = plt.subplots(ncols = 7, nrows = 2, figsize = (20, 10))
index = 0
ax = ax.flatten()

for col, value in df.items():
    sns.boxplot(y=col, data=df, ax=ax[index])
    index += 1
plt.tight_layout(pad = 0.5, w_pad=0.7, h_pad=5.0)
```



```
# Min-Max normalization
# Here we are taking only 4 column for normalization because in this the value are too high as com
pare to others

cols = ['trtbps', 'chol', 'thalachh', 'oldpeak', 'age']
for col in cols:
    minimum = min(df[col])
    maximum = max(df[col])
    df[col] = (df[col] - minimum)/ (maximum - minimum)
```

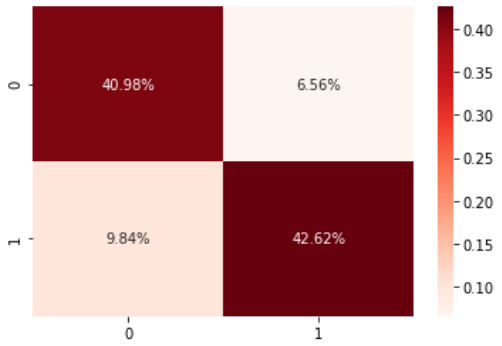
```
df.head()
```

	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
0	0.708333	1	3	0.481132	0.244292	1	0	0.603053	0	0.370968	0	0	1	1
1	0.166667	1	2	0.339623	0.283105	0	1	0.885496	0	0.564516	0	0	2	1
2	0.250000	0	1	0.339623	0.178082	0	0	0.770992	0	0.225806	2	0	2	1
3	0.562500	1	1	0.245283	0.251142	0	1	0.816794	0	0.129032	2	0	2	1
4	0.583333	0	0	0.245283	0.520548	0	1	0.702290	1	0.096774	2	0	2	1

Something that quite different was that he also calculated Type 1 and Type 2 error based on the predictions made to determine the accuracy of the model.

```
# confusion Maxtrix
cm1 = confusion_matrix(y_test, pred_lg)
sns.heatmap(cm1/np.sum(cm1), annot = True, fmt=  '0.2%', cmap = 'Reds')
```

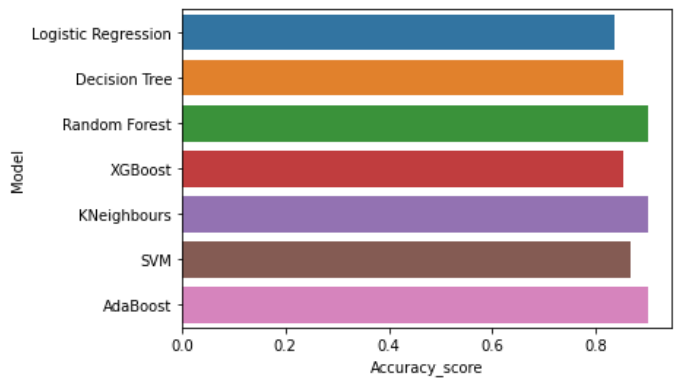
```
<AxesSubplot:>
```



- Here, Type-1 Error is 6.56% which is also known as False Positive.
- Type-2 Error is 9.84% which is also known as False Negative.
- while other % value in the confusion matrix represents that they are correctly predicted in their specific categories.

He has also plotted the performance of all the algorithms after doing hyper parameter tuning on them.

	Model	Accuracy_score
2	Random Forest	0.901639
4	KNeighbours	0.901639
6	AdaBoost	0.901639
5	SVM	0.868852
1	Decision Tree	0.852459
3	XGBoost	0.852459
0	Logistic Regression	0.836066



The link to his approach - https://www.kaggle.com/imakash3011/heart-attack-analysis-prediction-90-2

Another approach taken by Emre Arslan also used KNeighbors classification model but instead of using trial and error for finding the optimal 'number of neighbours (k)' hyperparameter, he used brute force method to calculate the accuracy of the model with different k values and picked the model with gave the highest accuracy percentage. We believe that we can use elbow method or silhouette plots method to quantify the optimal k value.

The link to his approach - https://www.kaggle.com/emrearslan123/eda-prediction-on-heart-attack-dataset